

Lab 05: Enterprise GitOps with Helm & ArgoCD

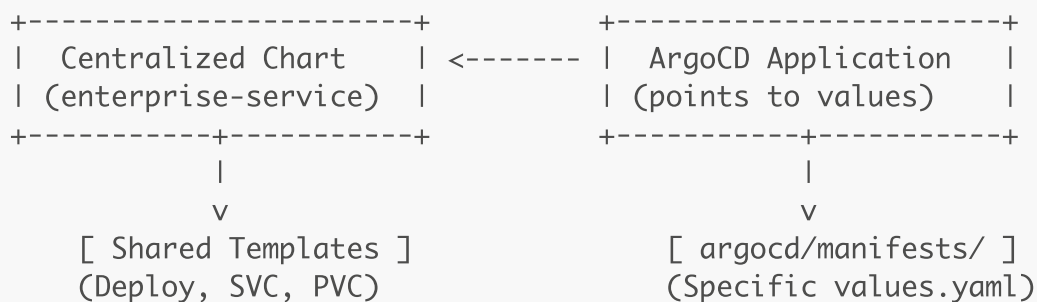
◇ 1. Git Repository Structure

This lab transitions from static YAML manifests to a **Centralized Helm Chart** pattern. Each microservice now uses a common base chart, providing its own **values.yaml**.

```
05-argocd-helm/
├── root-argocd.yaml          # Root Application (Helm-enabled)
├── charts/                   # Centralized Enterprise Chart
│   ├── enterprise-service/  # Base templates for all services
│   │   ├── Chart.yaml
│   │   ├── values.yaml      # Default global values
│   │   └── templates/       # Reusable Deployment, SVC, Secret
├── templates
│   ├── argocd/              # ArgoCD Child Manifests
│   │   ├── child-frontend-microservice.yaml
│   │   └── ...
│   ├── argocd/manifests/    # Service-specific Helm Charts
│   │   ├── frontend/
│   │   │   ├── Chart.yaml   # References ../../charts/enterprise-
│   │   │   └── values.yaml  # Frontend-specific overrides
│   └── ...
└── apps/                    # Application Source Code
    └── ...
```

◇ 2. Centralized Helm Architecture

In this improved design, we point all ArgoCD applications directly to the **Centralized Chart**, passing a specific **values.yaml** file for each microservice. This ensures naming consistency and avoids complex Helm subchart dependencies.

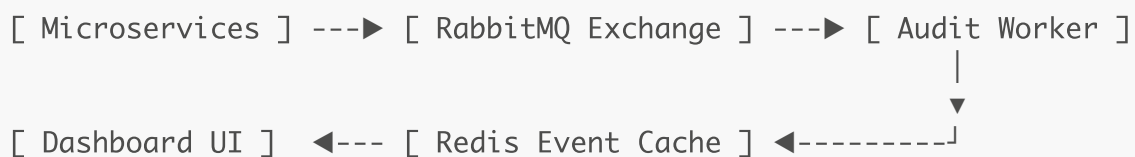


◇ 3. ArgoCD Configuration

Each microservice is defined as an ArgoCD **Application** that references the base chart and overrides values:

Example Application Source:

```
spec:
  source:
    path: charts/enterprise-service
    helm:
      valueFiles:
        - ../../argocd/manifests/login/values.yaml
```



◇ Enterprise Monitoring Features:

- **Live Audit Log:** Tracks "Transaction Paths" (e.g., **LOGIN** → **AUDIT_LOG**) in real-time.
- **Cache Health:** Displays live Redis memory usage and hit rates.
- **Database Metrics:** Automatic discovery of active tables and user counts.
- **Architecture-as-Code:** The Frontend renders **lab.md** directly into the UI for documentation accessibility.

◇ 4. Automated Build and Deploy

The provided automation script **rebuild_and_deploy.sh** has been updated to support the new Helm structure. It builds images, pushes them, and updates the **tag** in **values.yaml** for each service.

Usage:

```
# Run the script with a new tag (e.g., v10)
./rebuild_and_deploy.sh v10
```

◇ 5. Manual Build and Push (Detailed Steps)

Step 1: Login to Docker

```
docker login -u minjteck
```

Step 2: Build Microservices

1. Audit Service

```
cd apps/audit/app
docker build -t minjteck/audit-service:v1 .
docker push minjteck/audit-service:v1
cd ../../..
```

2. Frontend Service

```
cd apps/frontend/app
docker build -t minjteck/frontend-service:v1 .
docker push minjteck/frontend-service:v1
cd ../../..
```

3. Login Service

```
cd apps/login/app
docker build -t minjteck/login-service:v1 .
docker push minjteck/login-service:v1
cd ../../..
```

4. Register Service

```
cd apps/register/app
docker build -t minjteck/register-service:v1 .
docker push minjteck/register-service:v1
cd ../../..
```

5. Profile Service

```
cd apps/profile/app
docker build -t minjteck/profile-service:v1 .
docker push minjteck/profile-service:v1
cd ../../..
```

6. Forgot Password Service

```
cd apps/forgot-password/app
docker build -t minjteck/forgot-password-service:v1 .
docker push minjteck/forgot-password-service:v1
cd ../../..
```

7. Logout Service

```
cd apps/logout/app
docker build -t minjteck/logout-service:v1 .
docker push minjteck/logout-service:v1
cd ../../..
```

◇ 6. Deployment Steps (ArgoCD)

Deploying the enterprise stack with ArgoCD is a two-step process: initializing the root controller and letting it cascade the child microservices.

Step 1: Install Helm (Optional)

If you want to test the charts locally, install Helm:

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
| bash
```

Step 2: Install ArgoCD (Stable)

```
kubectl create namespace argocd
kubectl apply -n argocd -f
https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
```

Step 3: Deploy Root Application

The Root application manages all microservices. Applying this file will trigger ArgoCD to create 10 child applications. **ArgoCD will automatically handle the Helm dependency builds.**

```
kubectl apply -f root-argocd.yaml
```

Step 4: Access ArgoCD UI

```
# Get initial admin password
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" | base64 -d; echo

# Port forward to localhost:8080
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

Step 5: Verify Sync Status

Once the root app is synced, you should see 10 child applications in the ArgoCD UI. Each will point to `enterprise-lab` namespace.

◇ 7. Troubleshooting Common Issues

✗ Error: provided port is already allocated (30007/30008)

This means another service in the cluster is already using the port.

1. **Resolution:** We have updated the `frontend-service` to use **NodePort 30009** to avoid conflicts.
2. **Access URL:** Your new access URL will be `http://<NodeIP>:30009`.

✗ Error: spec.selector: field is immutable

This occurs when you change the labels or `fullnameOverride` of a microservice that was already deployed.

1. **Resolution:** Manually delete the problematic deployment and let ArgoCD recreate it with the correct selectors:

```
kubectl delete deployment <deployment-name> -n enterprise-lab
```

2. **Example:** `kubectl delete deployment mysql-service -n enterprise-lab`

✗ Internal Server Error (500) on Dashboard

This usually occurs because of a mismatch between the infrastructure configuration and what the app expects.

1. **Service Names:** Ensure all services use the `-svc` suffix as expected by the app (e.g., `mysql-svc`, `redis-svc`).
2. **Dependencies Not Ready:** If you see `waiting for mysql` in logs, it means the `initContainer` is doing its job. Ensure the `mysql-svc` pod is Running and Healthy.
3. **Check Logs:**

```
kubectl logs -f deployment/frontend-service-svc -n enterprise-lab
```

◇ 8. Manifest Architecture & Usage

Each microservice is bundled with specific Kubernetes objects that enable enterprise features:

Manifest	Purpose	Application Usage
Deployment	Pod Management	Defines the Python container, environment variables (MYSQL_HOST), and Liveness Probes (/health).
Service	Load Balancing	Provides internal DNS names like login-service which the Frontend uses for API calls. Frontend NodePort is 30009 .
Secret	Security	Holds sensitive DB passwords and Flask SECRET_KEY , injected as environment variables.
ConfigMap	Configuration	Stores non-sensitive data like DB names or RabbitMQ URLs.
PVC / SC	Persistence	Ensures MySQL and Redis data survives pod restarts. Used for the /var/lib/mysql mount point.
RBAC	Permissions	(Frontend Only) Allows the Python code to query kubectl style data from within the pod.

◇ 8. Validation

Default Admin Credentials

Use the following credentials to log into the web application:

- **Username:** **admin**
- **Email:** **admin@devops.com**
- **Password:** **admin**

Check Real-time Events

To see the RabbitMQ communication in action:

```
kubectl logs -f deployment/audit-service -n enterprise-lab
```

Now, go to the Frontend UI, login or register, and you will see the **[AUDIT]** logs appear instantly in your terminal!

◇ 9. Cleanup

To remove all resources created by this lab, use the following commands:

1. Delete Root Application

```
# Delete the root application
kubectl delete -f root-argocd.yaml
```

2. Delete All Child Applications

If child applications are still running in the ArgoCD UI, delete them manually:

```
# Delete all applications in the argocd namespace
kubectl delete apps --all -n argocd
```

3. Force Delete (If apps are stuck)

Delete the root application

```
kubectl delete -f root-argocd.yaml
```

```
### 2. Delete All Child Applications
If child applications are still running in the ArgoCD UI, delete them
manually:
```bash
Delete all applications in the argocd namespace
kubectl delete apps --all -n argocd
```

## 3. Force Delete (If apps are stuck)

If applications are stuck in "Deleting" status, remove their finalizers to force delete:

```
Remove finalizers and delete all apps
kubectl get apps -n argocd -o name | xargs -I {} kubectl patch {} -n
argocd --type=merge -p '{"metadata":{"finalizers":null}}' && kubectl
delete apps --all -n argocd
```

## 4. Remove Namespace

```
(Optional) Delete the application namespace
kubectl delete namespace enterprise-lab
```